

Analyse statique de programme par interprétation abstraite et intégration dans la plateforme MOPSA

Stage de Master d'initiation à la recherche
Été 2025–2026

Encadrant : Antoine Miné (antoine.mine@lip6.fr)
Durée : 2 mois (juin et juillet 2026)
Lieu du stage : Laboratoire LIP6, équipe APR,
Sorbonne Université, Paris, France

But. Le but du stage est de s'initier à la méthode d'analyse statique par interprétation abstraite, utilisée pour vérifier la correction des programmes. Il comprendra un travail bibliographique, un travail de développement formel ainsi qu'une partie d'implantation et d'expérimentation au sein de la plateforme d'analyse MOPSA développée dans l'équipe APR.

Contexte

L'analyse statique sémantique par interprétation abstraite permet de déterminer à la compilation des propriétés vraies de toutes les exécutions, et prouver formellement l'absence de certaines classes d'erreurs. Elle a permis le développement d'outils industriels, comme les analyseurs Astrée et Frama-C, qui ont notamment été appliqués avec succès à la vérification d'absence d'erreurs à l'exécution dans des logiciels embarqués critiques en C. Afin d'être efficaces, ces analyses calculent des représentations symboliques qui sûr-approximent l'ensemble des états accessibles du programme. Dans un contexte de validation, l'analyse doit être sûre (n'omettre aucun comportement vis à vis de la sémantique du programme, donc sans faux négatifs) et précise (peu de faux positifs). La sûreté est garantie par le cadre formel de l'interprétation abstraite [1].

L'équipe APR du LIP6 développe une plateforme d'analyse, MOPSA [3], qui est un logiciel libre programmé en OCaml fonctionnant sur ce principe. Cette plateforme fournit une architecture modulaire et des composants réutilisables, dont en particulier : une structure d'arbre syntaxique universelle et extensible, des itérateurs intra- et inter-procéduraux, des méthodes de calcul de point-fixes utiles pour l'analyse des boucles, des domaines abstraits pour les propriétés numériques, de mémoire et des objets, et un mécanisme flexible de communication et de réduction entre ces domaines. MOPSA intègre une analyse de C [4] et une analyse de Python [5] pour des propriétés d'absence d'erreur à l'exécution. Elle supporte également des travaux de recherche dans des directions originales, comme l'analyse des propriétés de sécurité [6], l'analyse de portabilité [7], l'analyse de *patches* [8], l'analyse de *smart contracts* [9] ou l'analyse de programmes OCaml [10].

Il est prévu d'étendre l'analyseur, d'un côté en proposant de nouveaux domaines pour améliorer la précision et l'efficacité de l'analyse sur les langages et les propriétés déjà supportées et, d'un autre côté, pour ajouter un support pour de nouveaux langages et de nouvelles propriétés qui bénéficieraient automatiquement des abstractions déjà présentes. Un autre but du projet est de faciliter l'enseignement de l'interprétation abstraite en fournissant une plateforme adaptée au développement de projets par les étudiants sur des langages à la sémantique simplifiée, afin de se familiariser avec les concepts fondamentaux de l'analyse statique.

Travail attendu

Le travail attendu pendant le stage comportera :

- une étude bibliographique ayant pour but de s’initier à l’interprétation abstraite sur le plan théorique [1, 2], puis à son instanciation au sein du projet MOPSA [3] ;
- une familiarisation à l’utilisation de la plateforme MOPSA pour le C et le Python ;
- l’étude de son architecture et de son implantation par lecture du code source ;
- le choix justifié d’un composant à ajouter à MOPSA ; il peut s’agit d’intégrer le support pour un nouveau langage de programmation, de nouvelles abstractions, d’une amélioration de la stratégie d’itération, etc. ;
- l’étude de l’état de l’art sur le type d’extension choisie ;
- l’implantation dans MOPSA de cette extension et l’expérimentation sur des cas d’exemple simples.

Références

- [1] P. Cousot and R. Cousot. Abstract interpretation : a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Conf. Rec. of the 4th Annual ACM SIGPLAN-SIGACT Symp. on Principles of Programming Languages (POPL’77)*, pp. 238–252. ACM Press, 1977.
- [2] A. Miné. Tutorial on static inference of numeric invariants by abstract interpretation. *Foundations and Trends in Programming Languages (FnTPL)*, 4(3–4) :120–372, 2017. <http://www-apr.lip6.fr/~mine/publi/article-mine-FTiPL17.pdf>.
- [3] M. Journault, A. Miné, R. Monat, and A. Ouadjaout. Combinations of reusable abstract domains for a multilingual static analyzer. In *Proc. of the 11th Working Conference on Verified Software : Theories, Tools, and Experiments (VSTTE19)*, volume 12031 of *Lecture Notes in Computer Science (LNCS)*, pages 1–18. Springer, Jul. 2019. <https://perso.lip6.fr/Antoine.Mine/publi/article-mine-al-vstte19.pdf>.
- [4] A. Ouadjaout and A. Miné. A library modeling language for the static analysis of C programs. In *Proc. of the 27th International Static Analysis Symposium (SAS’20)*, volume 12389 of *Lecture Notes in Computer Science (LNCS)*, pages 223–246. Springer, Nov. 2020. <https://perso.lip6.fr/Antoine.Mine/publi/ouadjaout-al-sas20.pdf>.
- [5] R. Monat, A. Ouadjaout, and A. Miné. A multilanguage static analysis of Python programs with native C extensions. In *Proc. of the 28th International Static Analysis Symposium (SAS’21)*, volume 12913 of *Lecture Notes in Computer Science (LNCS)*, pages 323–345. Springer, Oct. 2021. <https://perso.lip6.fr/Antoine.Mine/publi/article-monat-al-sas21.pdf>.
- [6] F. Parolini and A. Miné. Sound abstract nonexploitability analysis. In *Proc. of the 25th International Conference on Verification, Model Checking, and Abstract Interpretation (VMCAI’24)*, volume 14499 of *Lecture Notes in Computer Science (LNCS)*. Springer, Jan. 2024. <https://perso.lip6.fr/Antoine.Mine/publi/article-parolini-al-vmcai24.pdf>.
- [7] D. Delmas, A. Ouadjaout, and A. Miné. Static analysis of endian portability by abstract interpretation. In *Proc. of the 28th International Static Analysis Symposium (SAS’21)*, volume 12913 of *Lecture Notes in Computer Science (LNCS)*, pages 102–123. Springer, Oct. 2021. <https://perso.lip6.fr/Antoine.Mine/publi/article-delmas-al-sas21.pdf>.
- [8] D. Delmas and A. Miné. Analysis of software patches using numerical abstract interpretation. In *Proc. of the 26th International Static Analysis Symposium (SAS’19)*, volume 11822 of *Lecture Notes in Computer Science (LNCS)*, pages 225–246. Springer, Oct. 2019. <https://perso.lip6.fr/Antoine.Mine/publi/article-delmas-mine-sas19.pdf>.
- [9] G. Bau, A. Miné, V. Botbol, and M. Bouaziz. Abstract interpretation of Michelson smart-contracts. In *Proc. of the 11th ACM SIGPLAN International Workshop on the State Of the Art in Program Analysis (SOAP’22)*, pages 36–43. ACM, Jun. 2022. <https://perso.lip6.fr/Antoine.Mine/publi/article-bau-al-soap2022.pdf>.
- [10] M. Valnet, R. Monat, and A. Miné. Compositional static value analysis for higher-order numerical programs. In *Proc. of the 39th European Conference on Object-Oriented Programming (ECOOP 2025)*, volume 333 of *Leibniz International Proceedings in Informatics (LIPIcs)*, page 29. Dagstuhl Publishing, Jun. 2025. <https://perso.lip6.fr/Antoine.Mine/publi/article-valnet-al-ecoop25.pdf>.